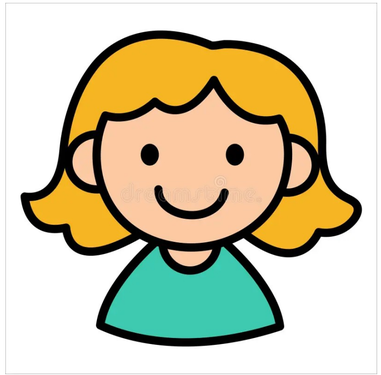
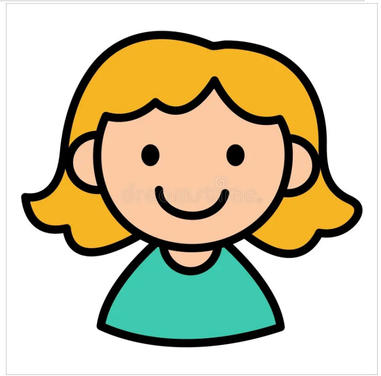
Engineering intuition behind BIP352, BIP374

@stratospher on 14/10/2025

Alice sends bitcoins to Bob





Address reuse is bad!

Bob’s donation page



My bitcoin address: B

Bob should generates new address to receive every time

Bob can use btcpayserver



Silent payment

- static addresses
- transaction identified only by sender + receiver


Refresher!



Bitcoin address

an address is just a user-friendly encoding of a locking script.

(script hash, public key hash, tweaked public key)

Refresher!

Normal bitcoin addresses

P2WPKH

Bech32

Random Example

Example P2WPKH Address:

`bc1q85pfvswxn48ljnnt3j0htk0up70samr2kckny8`

42 characters

Example P2WPKH ScriptPubKey:

ASM | Hex

```
OP_0
OP_PUSHBYTES_20
3d029641c69d4ff94e6b8c9f75d9fc0f9f0eec6a
```

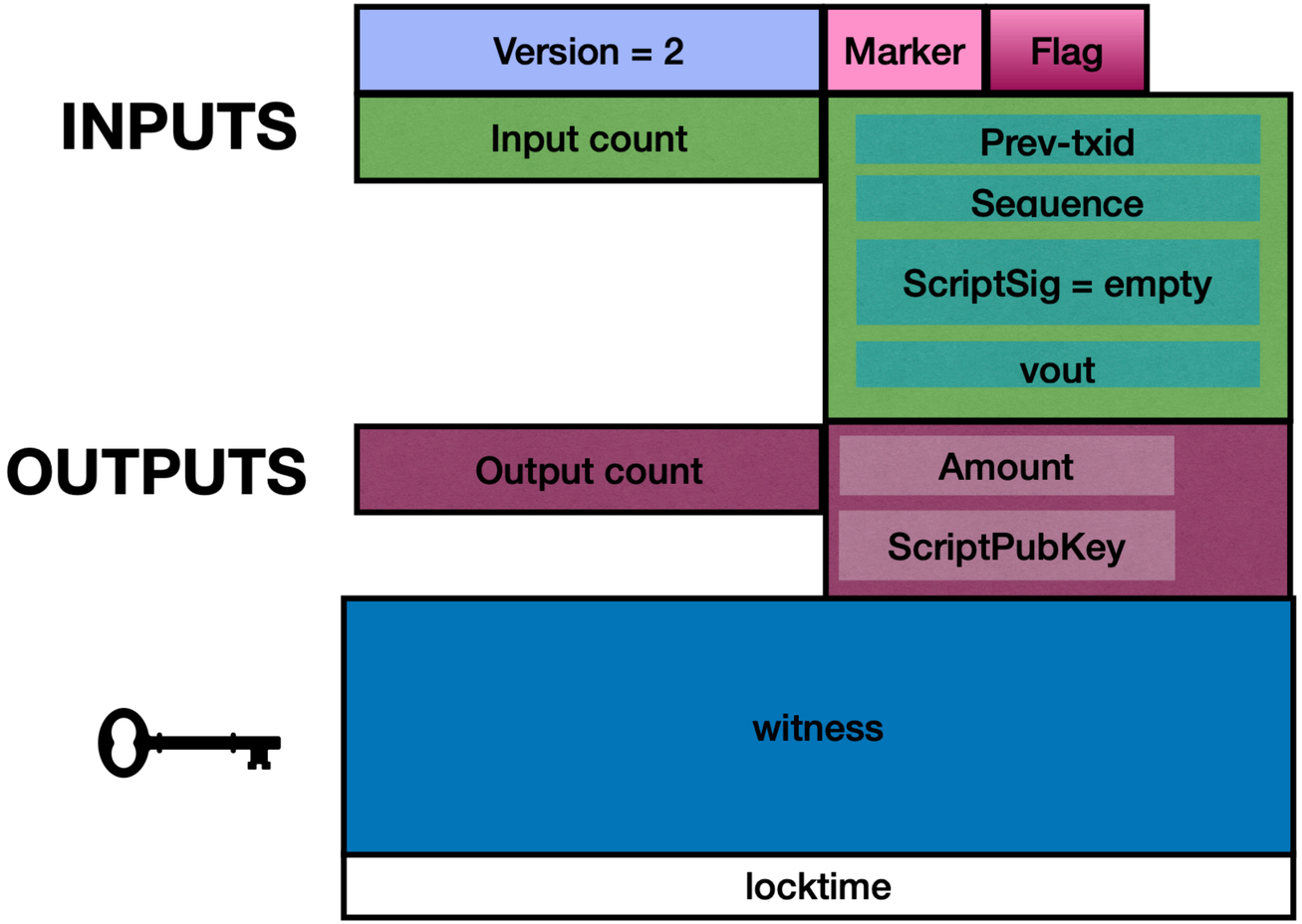
A 42-character `bc1qaddress` contains a `public key hash` and corresponds to a P2WPKH locking script.

Refresher!

Normal bitcoin transaction

P2WPKH

ScriptPubKey = locking script



ScriptPubKey

OP_0 <pubkey_hash>

Refresher!

Normal bitcoin transaction

P2TR

Bech32m

Random Example

Example P2TR Address:

`bc1p0yksrxszvr6th2cxwld6wwhk5jg9wuh3cvp2mv9mdjfzaugsr7nsfr8845`

62 characters

Example P2TR ScriptPubKey:

ASM | Hex

```
OP_1
OP_PUSHBYTES_32
07cd1a2b906e4eaf1cc45d177b2cd5fb9ef63fc5fc41e934d7ede4dc7ea1434c
```

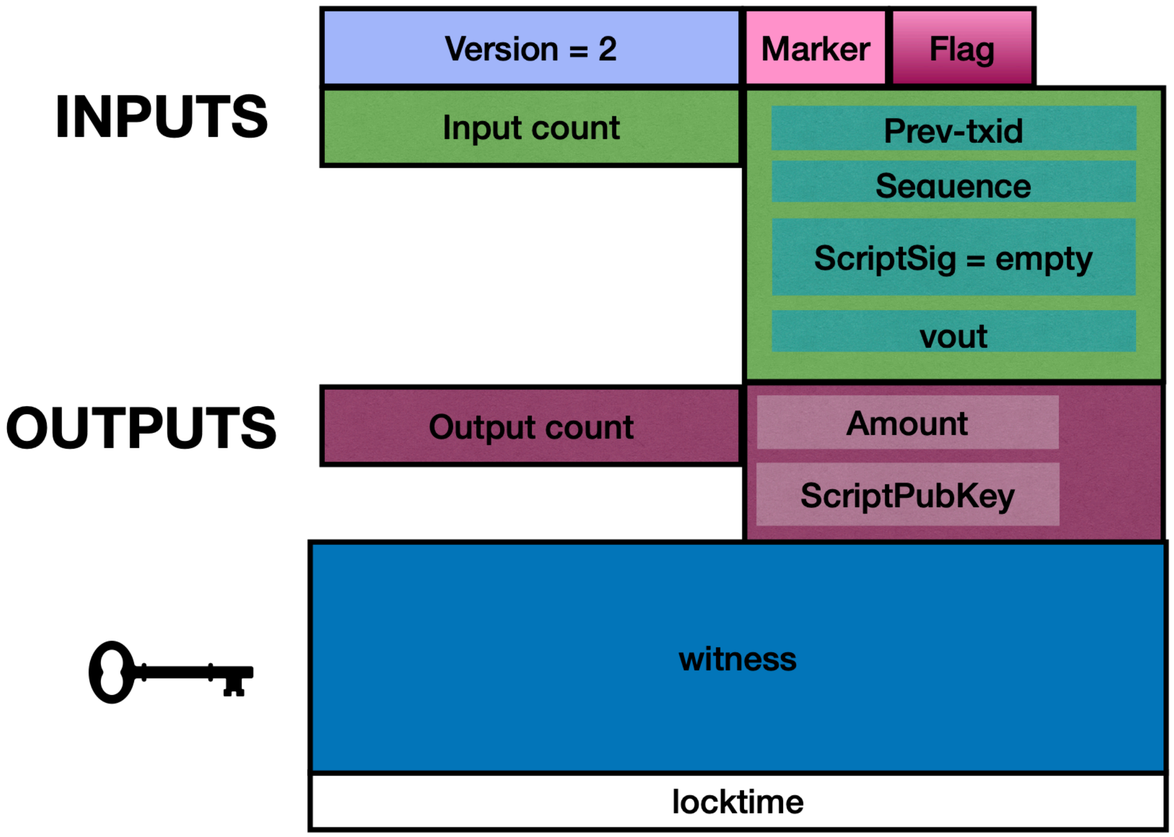
A 62-character **bc1p**address contains a *tweaked public key* and corresponds to a P2TR locking script.

Refresher!

Normal bitcoin addresses

P2TR

ScriptPubKey = locking script



ScriptPubKey

OP_1 <32 byte tweakpubkey>

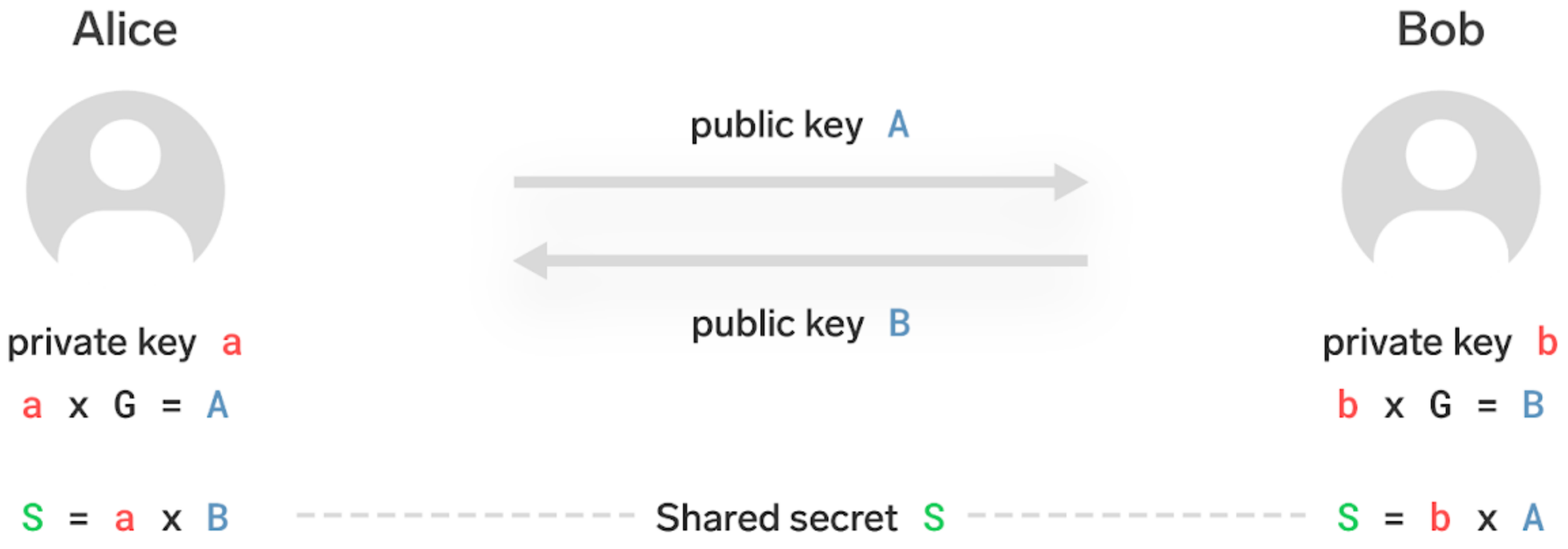


Basic cryptography





Basic cryptography



Attempt 1: Alice wants to send



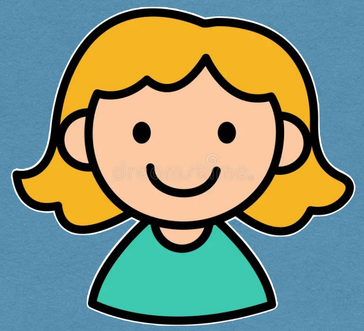
Alice computes P = B + hash(a·B)·G

Bob’s donation page



My silent payment address: B

Attempt 1: Bob wants to receive



Alice computes P = B + hash(a·B)·G

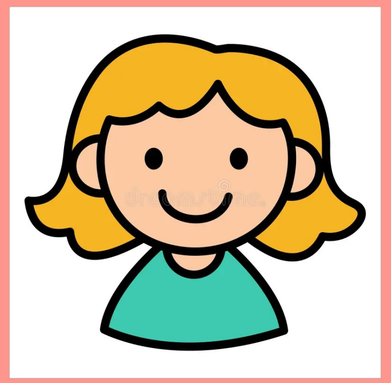


Bob scans

for every unspent taproot output:

Bob computes P = B + hash(b·A)·G

Attempt 2: Alice wants to send many times



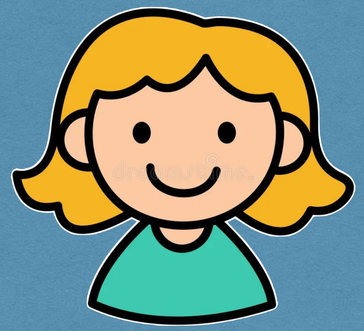
Alice computes P = B + hash(a·B || k)·G

Bob’s donation page

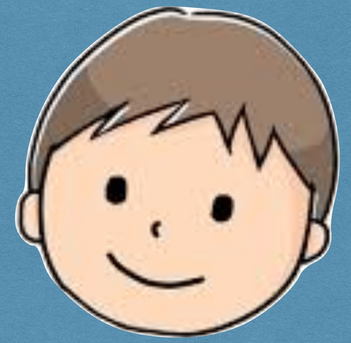


My silent payment address: B

Attempt 2: Bob wants to receive many times



Alice computes $P = B + \text{hash}(a \cdot B \,\|\, k) \cdot G$

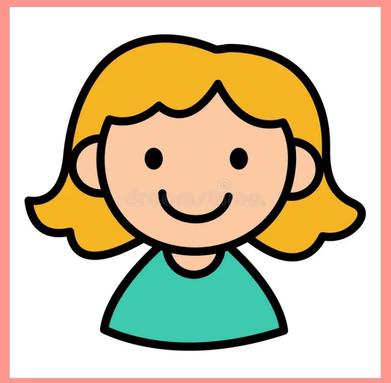


Bob scans

for every unspent taproot output:

Bob computes $P = B + \text{hash}(b \cdot A \,\|\, k) \cdot G$

Attempt 3: Alice wants to send many times using different UTXOs



where input_hash = hash(outpoint || A)

Alice computes P = B + hash(input_hash.a·B || k)·G

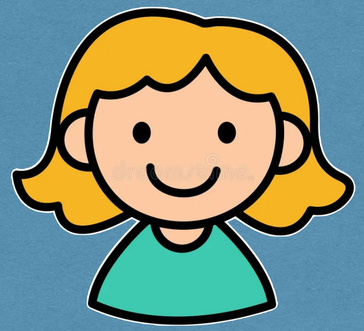
Bob’s donation page



My silent payment address: B

Alice computes P = B + hash(input_hash’.a’·B || k)·G and chooses a’ such that a’ = input_hash.a /input_hash
So now P = B + hash(input_hash.a·B || k)·G

Attempt 3: Bob wants to receive many times With Alice's different UTXO





where input_hash = hash(outpoint || A)

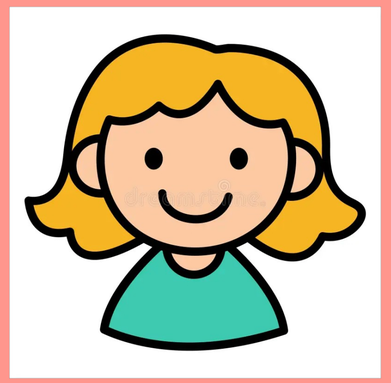
Alice computes P = B + hash(input_hash.a·B || k)·G

Bob scans

for every unspent taproot output:

Bob computes P = B + hash(input_hash.b·A || k)·G

Attempt 4: Alice wants to send many times using different UTXOs + tx also has other inputs



sum of Alice public keys

Smallest

where input_hash = hash(outpoint_L || A)

Alice computes P = B + hash(input_hash·a·B || k)·G

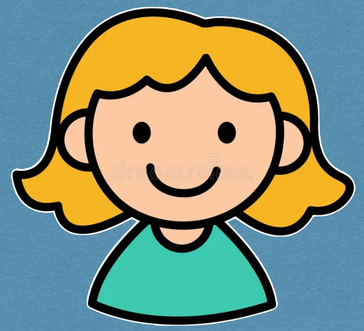
sum of Alice private keys

Bob’s donation page



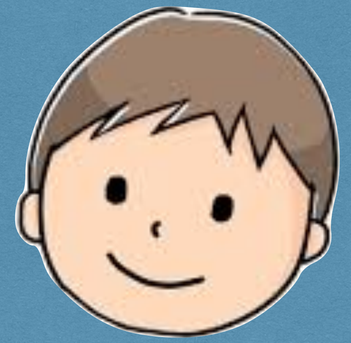
My silent payment address: B

Attempt 4: Bob wants to receive many times
With Alice's different UTXO + tx also has other inputs



where input_hash = hash(outpoint_L || A)

Alice computes P = B + hash(input_hash.a·B || k)·G

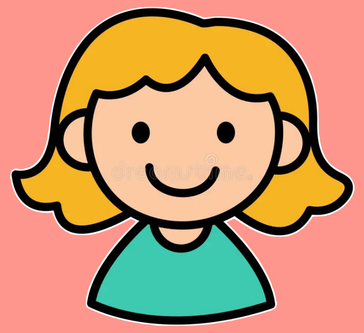


Bob scans

for every unspent taproot output:

Bob computes P = B + hash(input_hash.b·A || k)·G

sum of Alice public keys



sum of Alice public keys

Smallest

where input_hash = hash(outpoint_L || A)

Alice computes P = B + hash(input_hash·a·B || k)·G

sum of Alice private keys

scanning
means Bob’s private key always online !!!



Bob scans

for every unspent taproot output:

Bob computes P = B + hash(input_hash.b·A || k)·G

sum of Alice public keys

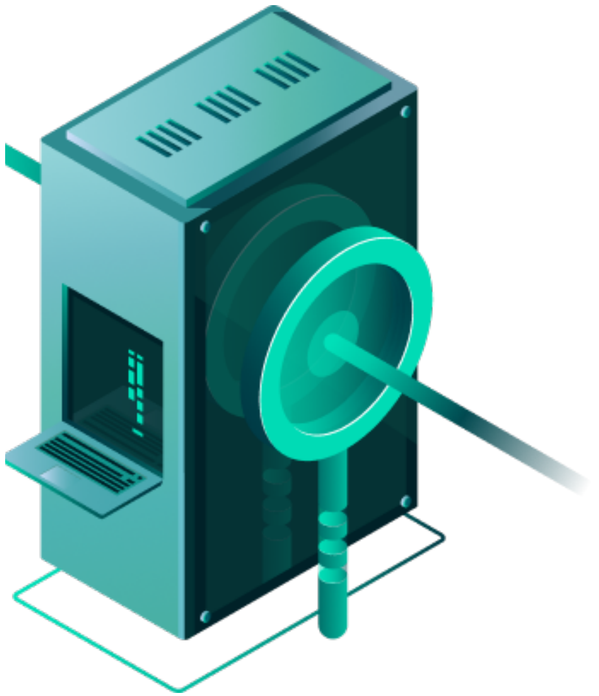
Bob can use 2 public keys in his address

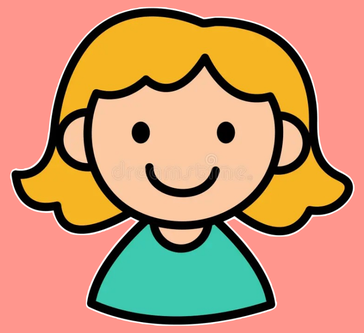


Bob’s donation page



My silent payment address: (B_spend, B_scan)





where input_hash = hash(outpoint_L || A)

Alice computes $P = B_{spend} + hash(input_hash.a \cdot B_{scan} || k) \cdot G$



scanning
means Bob’s private key always online

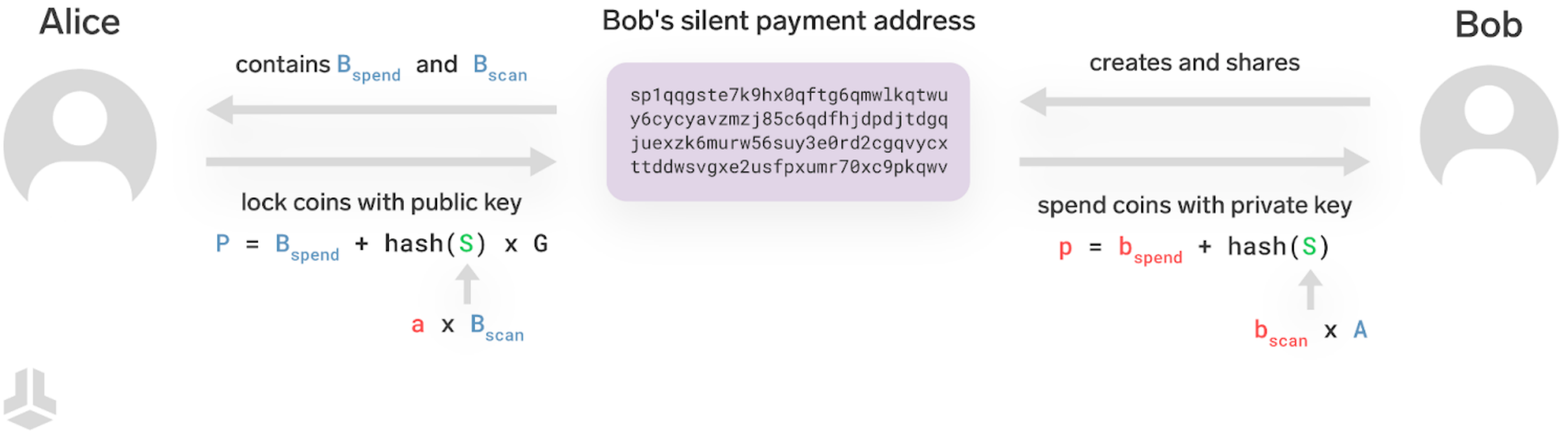
Bob scans

for every unspent taproot output:

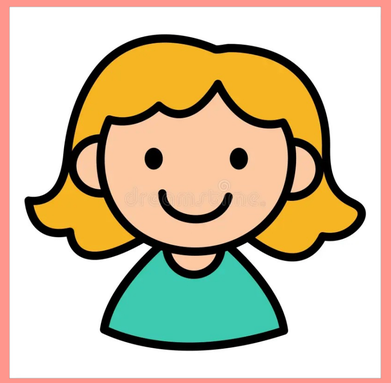
where input_hash = hash(outpoint_L || A)

Bob computes $P = B_{spend} + hash(input_hash.A.b_{scan} || k)\cdot G$

Bob want to spend with privat key = $b_{spend} + hash(input_hash.A.b_{scan} || k)\cdot G$


Alice
contains B_{spend} and B_{scan}
lock coins with public key
$P = B_{spend} + hash(S) \times G$
$a \times B_{scan}$
Bob's silent payment address
sp1qqgste7k9hx0qftg6qmwlkqtwu
y6cycyavzmzj85c6qdfhjdpdjtdgq
juexzk6murw56suy3e0rd2cgqvycx
ttddwsvgxe2usfpxumr70xc9pkqwv
creates and shares
spend coins with private key
$p = b_{spend} + hash(S)$
$b_{scan} \times A$
Bob

Attempt 5: Alice wants to send many times using different UTXOs + tx also has other inputs + Bob wants to label his addresses



where input_hash = hash(outpoint_L || A)

Alice computes P = B_m + hash(input_hash.a·B_scan || k)·G

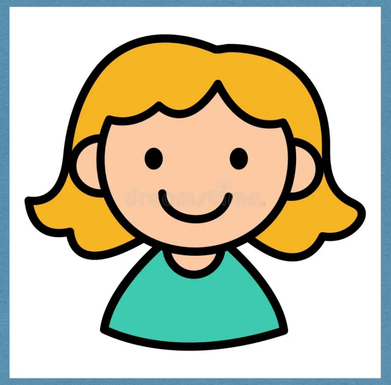
Bob's donation page



My silent payment address: (B_m, B_scan)

Nothing changes for Alice: uses whatever labelled spend key Bob publishes

Attempt 5: Bob wants to receive many times
With Alice's different UTXO + Alice tx also has other inputs
+ Bob wants to label his addresses





where input_hash = hash(outpoint_L || A)

Alice computes P = B_m + hash(input_hash.a·B_scan || k)·G

Bob scans

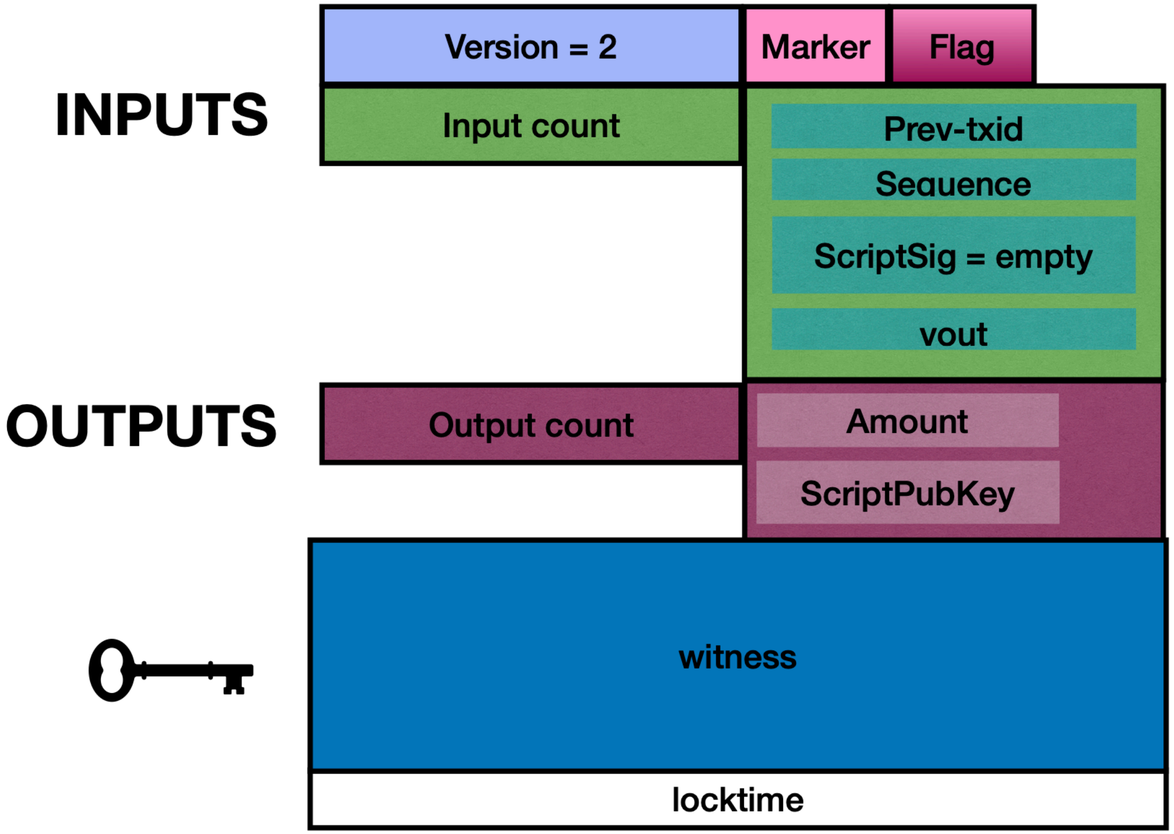
for every unspent taproot output:

Bob computes P = B_spend + hash(b_scan || m)·G + hash(input_hash.A·b_scan || k)·G

Normal bitcoin addresses

P2TR

ScriptPubKey = locking script





ScriptPubKey

OP_1 <32 byte tweakpubkey>

Part 2 : DLEQ proofs

given:

elliptic curve points A, B, C, G + scalar a known only to the prover

where $A = a \cdot G$ and $C = a \cdot B$

the prover proves knowledge of a without revealing anything about a.



Alice private key + public key

a

A



Bob public key

B



Shared secret

C

G



Makes proof

proof

She knows Alice private key!

VERIFY

A

B

C

G

Why?



where input_hash = hash(outpoint_L || A)

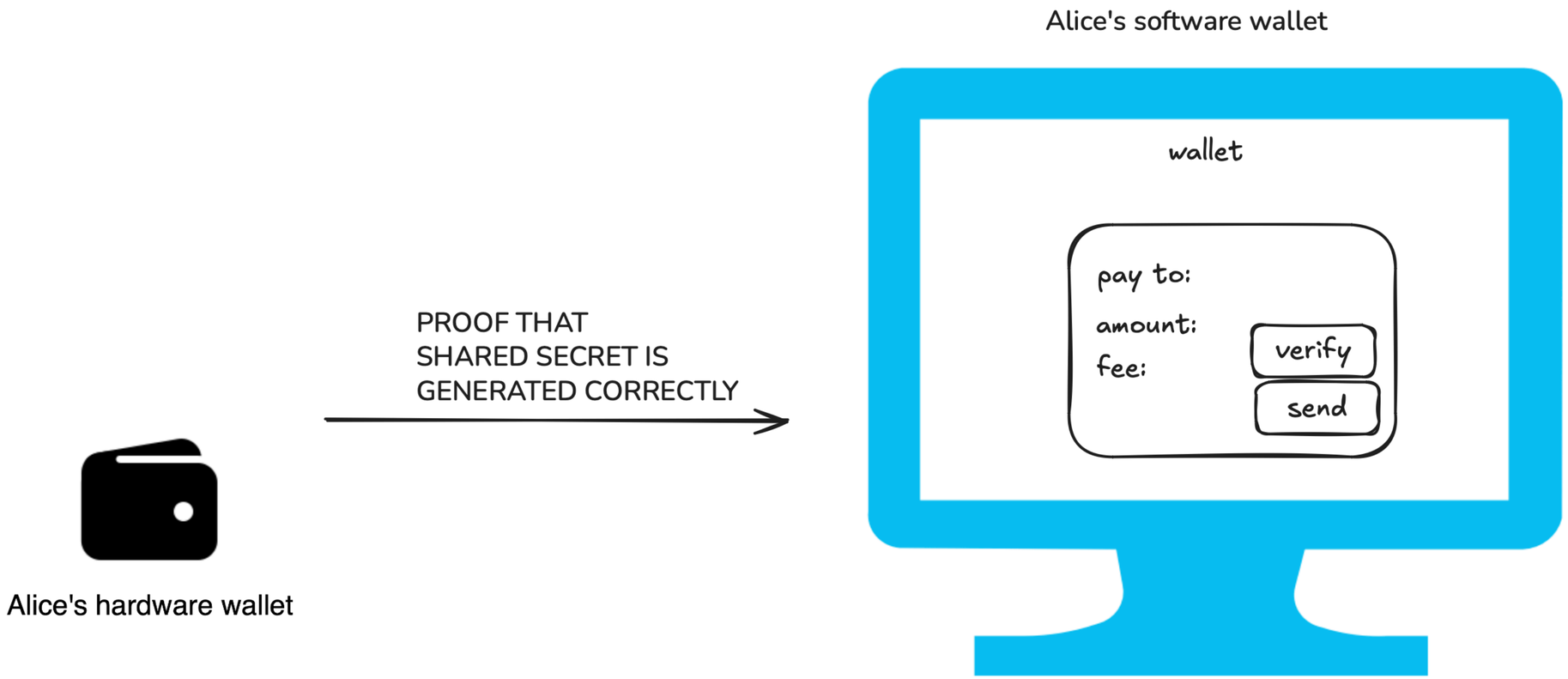
Alice computes $P = B_{spend} + hash(input_hash.a \cdot B_{scan} || k) \cdot G$

Requires private key

Funds can get lost if there is computed incorrectly

Alice's signer who has her private key would like to prove: "shared secret is correct" without revealing Alice's private key

Alice’s signer who has her private key would like to prove: “shared secret is correct” without revealing Alice’s private key



PROOF THAT I KNOW
ALICE PRIVATE KEY

⟹

PROOF THAT
SHARED SECRET IS
GENERATED CORRECTLY

given:

elliptic curve points A, B, C, G + scalar a known only to the Alice's signer (the prover)

where A = a · G, B = B_scan, C = input_hash. a · B_scan

the prover proves knowledge of a without revealing anything about a.

"Proves knowledge" WILL IMPLY "correctness" here

Prover



Alice private key + public key

a A



Bob public key

B



Shared secret

C

G

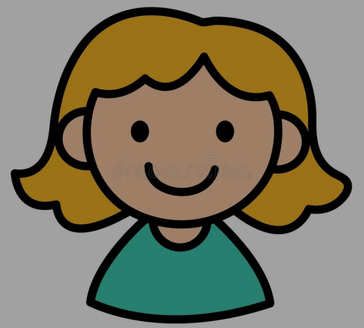
Given: A = a.G, B = B_scan, C = input_hash.a.B_scan + a which is only known to signer

The basic proof generation uses a random scalar k, the secret a, and the point being proven $C = a \cdot B$.

- Let $R_1 = k \cdot G$.
- Let $R_2 = k \cdot B$.
- Let $e = hash(R_1 \,||\, R_2)$.
- Let $s = (k + e \cdot a)$.

Providing only C, e and s as a proof does not reveal a or k.

Prover





Alice private key + public key



Bob public key



Shared secret

a

A

B

C

G

INPUT

Generates random k

k

R1 = kG

R2 = kB

R1

R2

HASH

s = k + ea

e

s

OUTPUT

Verifier



Given: A = a.G, B = B_scan, C = input_hash.a.B_scan

Verifying the proof involves recreating R_1 and R_2 with only e and s as follows:

- Let $R_1 = s \cdot G - e \cdot A$.
- Let $R_2 = s \cdot B - e \cdot C$.

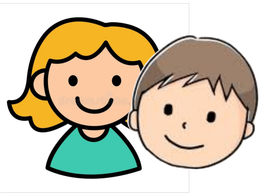
This can be verified by substituting $s = (k + e \cdot a)$:

- $s \cdot G - e \cdot A = (k + e \cdot a) \cdot G - e \cdot A = k \cdot G + e \cdot (a \cdot G) - e \cdot A = k \cdot G + e \cdot A - e \cdot A = k \cdot G$.
- $s \cdot B - e \cdot C = (k + e \cdot a) \cdot B - e \cdot C = k \cdot B + e \cdot (a \cdot B) - e \cdot C = k \cdot B + e \cdot C - e \cdot C = k \cdot B$.

Thus verifying $e = hash(R_1 \,||\, R_2)$ proves the discrete logarithm equivalency of A and C.







Alice public key

Bob public key

Shared secret

s

e

G

A

B

C

INPUT

R1

HASH

R2

e

e

==?

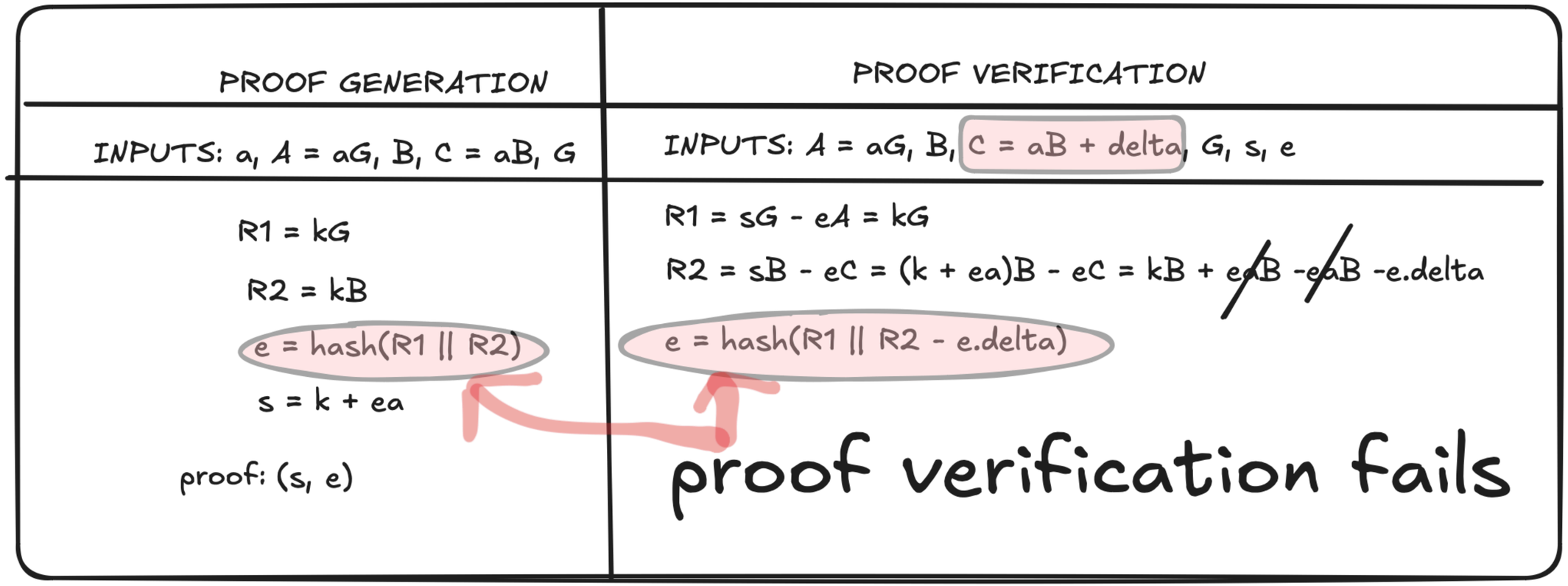
e

OUTPUT

Verifier



“Proof that I know Alice’s private key”



“Shared secret is generated correctly”

Use cases

1. Memory corruption

where input_hash = hash(outpoint_L || A)

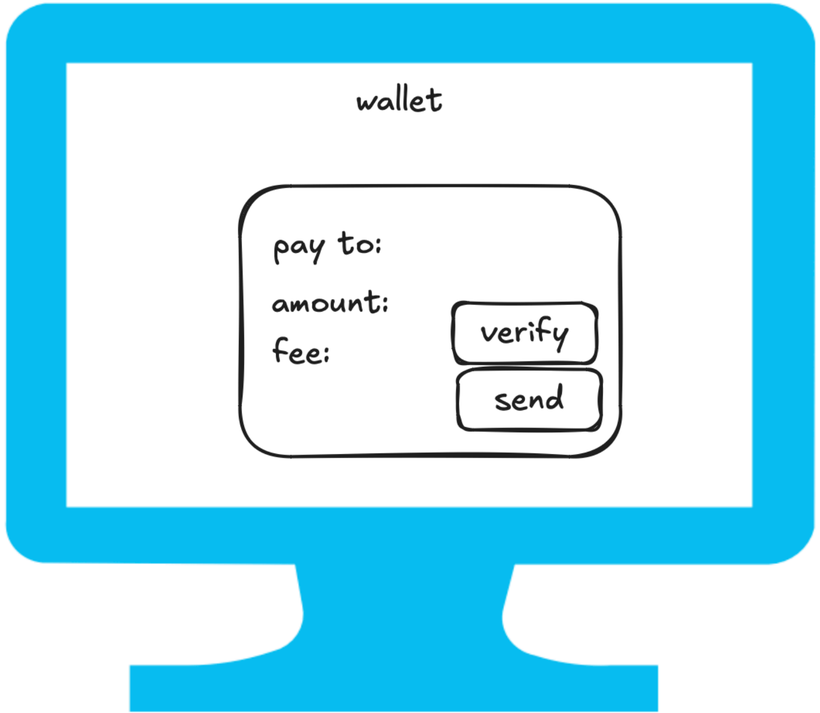
Alice computes $P = B_{spend} + hash(input_hash.a \cdot B_{scan} || k) \cdot G$

Requires private key



Funds can get lost if there is computed incorrectly

Alice's software wallet



PROOF THAT
SHARED SECRET IS
GENERATED CORRECTLY



Alice's hardware wallet